

Contents

1 To-Do App	1
1.0.1 Refined Database Schema	1
1.0.2 Additional Considerations	2
1.0.3 PHP OOP Approach	2
1.0.4 Example: PHP Class for Todo	2
1.0.5 Web Interface	3
1.0.6 Security	3

1 To-Do App

Database design for a To-Do App with user registration and role management, refined version of your database schema, including a table for categories and foreign key constraints for relational integrity:

1.0.1 Refined Database Schema

- Table: todos

```
CREATE TABLE todos (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  user_id INT NOT NULL,  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  category_id INT,  
  done BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id),  
  FOREIGN KEY (category_id) REFERENCES categories(id)  
);
```

- Table: users

```
CREATE TABLE users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(255) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  role VARCHAR(255) NOT NULL DEFAULT 'user'  
);
```

- Table: categories

```
CREATE TABLE categories (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(255) NOT NULL UNIQUE  
);
```

1.0.2 Additional Considerations

1. **Categories Table:** A separate table for categories allows more flexibility and normalization. Users can create custom categories, or you can pre-populate it with common categories.
2. **Foreign Keys:** These ensure relational integrity. For example, `user_id` in the `todos` table references `id` in the `users` table.
3. **Timestamps:** Consider automatically updating the `updated_at` timestamp. In MySQL, you can use `ON UPDATE CURRENT_TIMESTAMP`.
4. **Password Storage:** Ensure you store hashed passwords, not plain text. Use PHP's `password_hash()` and `password_verify()` for handling passwords.
5. **Indexes:** Depending on your queries, consider adding indexes to frequently searched fields for performance optimization.
6. **Roles Management:** If your application's roles have different permissions or capabilities, consider having a separate `roles` table and link it to the `users` table. This allows for easier management and scalability of roles.

1.0.3 PHP OOP Approach

For the PHP backend, consider the following OOP structure:

- **Classes for Database Tables:** Create classes like `Todo`, `User`, `Category` each encapsulating CRUD operations for their respective tables.
- **Session Management:** Use PHP sessions for handling user logins.
- **Authentication and Authorization:** Implement methods in your `User` class for handling user authentication and role-based authorization.
- **Validation and Sanitization:** Include methods for validating and sanitizing input data to ensure security and data integrity.
- **Error Handling:** Robust error handling mechanisms to manage and respond to any exceptions or errors that occur during database operations.

1.0.4 Example: PHP Class for Todo

Here's a basic structure for the `Todo` class:

```
class Todo {
    private $pdo;

    public function __construct($pdo) {
        $this->pdo = $pdo;
    }

    public function create($userId, $title, $description, $categoryId) {
        // Insert a new todo into the database
    }
```

```
}

public function markDone($todoId) {
    // Mark a specific todo as done
}

// Additional CRUD methods...
}
```

Each class (Todo, User, Category) will follow a similar pattern, with methods for each CRUD operation.

1.0.5 Web Interface

For the web interface, you'll create HTML forms and PHP scripts to interact with these classes. For example, forms for adding a todo, user registration, login, etc., and corresponding PHP scripts to process these forms and call the appropriate class methods.

1.0.6 Security

- Use prepared statements to prevent SQL injection.
- Implement CSRF tokens in forms to prevent cross-site request forgery.
- Hash passwords using `password_hash()` for storage.
- Implement proper session management for user authentication.

By combining these database structures and OOP principles in PHP, you'll create a robust, scalable, and secure To-Do application.